

REST API - Cheat Sheet

REST (Representational State Transfer) is an architectural style for designing networked APIs. Most modern web APIs claim to be REST-based, though many bend the rules. The core ideas help you design APIs that other developers find predictable, debuggable, and easy to consume.

What REST Is

REST treats everything as a **resource** identified by a URL. You interact with resources using a small set of HTTP methods. The server doesn't keep client session state between requests.

A REST API typically:

- Uses HTTP as the transport.
 - Identifies resources by URL. Example: `/users/42` for user 42.
 - Uses HTTP methods to express the operation (GET, POST, PUT, DELETE).
 - Returns data in JSON (the common default today) or sometimes XML.
 - Returns HTTP status codes to indicate success or failure.
-

The REST Constraints

Roy Fielding's PhD dissertation (2000) listed six constraints. The ones that matter day-to-day:

- **Client-server:** clean separation between the UI and data storage.
- **Stateless:** each request carries everything needed. No server-side session.
- **Cacheable:** responses say whether they can be cached (`Cache-Control` header).
- **Uniform interface:** consistent resource URLs, standard HTTP methods, self-descriptive messages.
- **Layered system:** clients can't tell if they're talking to the origin server or a proxy.

The sixth (code on demand) is optional and rarely used. Most APIs people call "REST" don't satisfy all six strictly. Fielding himself called these "REST-ish" or just HTTP APIs.

HTTP Methods

Method	Use	Safe?	Idempotent?
GET	Retrieve a resource	Yes	Yes
POST	Create a new resource (or non-idempotent action)	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partial update of a resource	No	Often yes
DELETE	Remove a resource	No	Yes
HEAD	Same as GET but no body	Yes	Yes
OPTIONS	What methods does this URL support?	Yes	Yes

Safe means the operation doesn't change server state. **Idempotent** means calling it multiple times has the same effect as calling it once.

HTTP Status Codes

Code	Meaning
200 OK	Standard success
201 Created	Resource created (used with POST)
202 Accepted	Request accepted, processing later
204 No Content	Success with no body (used with DELETE, sometimes PUT)
301 Moved Permanently	URL has changed forever
302 Found	Temporary redirect
304 Not Modified	Cached response is still valid
400 Bad Request	Client sent something malformed
401 Unauthorized	Not authenticated
403 Forbidden	Authenticated but not allowed
404 Not Found	Resource doesn't exist
405 Method Not Allowed	Wrong HTTP method for this URL
409 Conflict	Resource state prevents the operation
422 Unprocessable Entity	Valid syntax but semantic errors
429 Too Many Requests	Rate limit hit
500 Internal Server Error	Something broke on the server
502 Bad Gateway	Upstream service returned an invalid response
503 Service Unavailable	Server is down or overloaded
504 Gateway Timeout	Upstream service didn't respond in time

The general buckets:

- **2xx:** success
- **3xx:** redirect
- **4xx:** client error (you messed up)
- **5xx:** server error (we messed up)

Resource URL Design

URLs name resources. Use nouns and pluralize collections. Save verbs for HTTP methods.

Good:

```
GET    /users          # list users
GET    /users/42       # get user 42
POST   /users          # create a user
PUT    /users/42     # replace user 42
PATCH /users/42     # partially update user 42
DELETE /users/42     # delete user 42
```

Bad:

```
GET /getUsers
POST /createUser
POST /deleteUser?id=42
```

Nested resources:

```
GET /users/42/orders      # orders belonging to user 42
GET /users/42/orders/100  # order 100 belonging to user 42
```

Try to keep nesting at 2 levels max. URLs like `/users/42/orders/100/items/5/options/3` get painful for everyone.

Use kebab-case in URLs:

```
/blog-posts      # good
/blogPosts       # ugly
/blog_posts      # unusual for URLs
```

Request and Response Anatomy

Request

```
POST /users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
Accept: application/json

{
  "name": "alice",
  "email": "alice@example.com"
}
```

Parts:

- **Method:** POST
- **Path:** /users
- **Headers:** key-value metadata
- **Body:** JSON payload

Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /users/42

{
  "id": 42,
  "name": "alice",
  "email": "alice@example.com"
}
```

Parts:

- **Status code:** 201
 - **Headers:** including `Location` pointing to the new resource
 - **Body:** the created resource
-

Common Headers

Request headers

Header	Purpose
Authorization	Credentials (token, API key, basic auth)
Content-Type	Format of the request body (application/json)
Accept	Format the client wants back
User-Agent	Identifies the client
If-Match / If-None-Match	Conditional requests via ETag
X-Request-Id	Custom tracing ID

Response headers

Header	Purpose
Content-Type	Format of the response body
Location	URL of a newly created resource (with 201)
Cache-Control	How clients can cache the response
ETag	Version identifier for the resource
Retry-After	When the client can try again (with 429 or 503)

Authentication

Bearer Token (most common)

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Token is usually a JWT. Server validates the signature and extracts claims.

API Key

```
Authorization: Api-Key abc123
# or
X-API-Key: abc123
# or in query string (avoid, gets logged)
?api_key=abc123
```

Simple, no expiration unless you add one. Good for service-to-service.

Basic Auth

```
Authorization: Basic dXNlcjpwYXNz # base64(user:pass)
```

Only safe over HTTPS. Used internally or for simple admin endpoints.

OAuth 2.0

A flow for letting users authorize third-party access without sharing passwords. The client receives an access token (bearer) after the flow. Common flows:

- **Authorization code:** server-side web apps.
- **PKCE:** mobile apps and SPAs.
- **Client credentials:** service-to-service.

Path, Query, Body, Headers

Each carries a different kind of data.

Location	Use for
Path	Identifying the resource (/users/42)
Query string	Filtering, sorting, pagination (? status=active&limit=20)
Body	Resource data (POST, PUT, PATCH payloads)
Headers	Metadata (auth, content type, tracing)

Rule of thumb: if it identifies what you want, it goes in the path. If it filters or modifies the result, it goes in the query string. If it's the data being sent, it goes in the body.

Versioning

Three common strategies.

1. URL versioning (most popular)

```
GET /v1/users/42
GET /v2/users/42
```

Simple, visible. Downside: clients need to update URLs when you bump versions.

2. Header versioning

```
GET /users/42
Accept: application/vnd.example.v2+json
```

Cleaner URLs. Downside: harder to debug, hidden from logs at a glance.

3. Query parameter versioning

```
GET /users/42?version=2
```

Easy to use but feels off because version isn't really a filter on the query.

Pick one and stick with it. URL versioning is the most common because it's easiest to debug, document, and route.

Pagination, Filtering, Sorting

Pagination

Two common styles.

Offset / limit:

```
GET /users?offset=20&limit=10
```

Easy to implement. Bad at large offsets because the DB still has to skip rows.

Cursor-based:

```
GET /users?cursor=abc123&limit=10
```

The cursor encodes the last-seen row. Handles large datasets well and stays consistent when data changes underneath.

Filtering

```
GET /users?status=active&role=admin
```

Multiple query params combine with AND. For complex filters, consider a dedicated query language or a `POST /users/search` endpoint that accepts a JSON body.

Sorting

```
GET /users?sort=created_at:desc,name:asc
```

Or use a `-` prefix for descending:

```
GET /users?sort=-created_at,name
```

Idempotency and Safety

- **Safe methods** don't change state: GET, HEAD, OPTIONS.
- **Idempotent methods** can be retried without side effects: GET, PUT, DELETE, HEAD, OPTIONS. PATCH is sometimes idempotent.
- **POST** is neither safe nor idempotent by default.

For non-idempotent operations that must be retried safely (payments, order creation), use an **idempotency key**:

```
POST /orders
Idempotency-Key: 8f3a-2bc4-...

{ "items": [...] }
```

The server stores the result keyed by the idempotency key. A retry with the same key returns the same response without doing the work twice.

Java / Spring Boot Example

A typical Spring Boot REST controller:

```

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService service;

    public UserController(UserService service) {
        this.service = service;
    }

    @GetMapping
    public List<User> list(
        @RequestParam(defaultValue = "0") int offset,
        @RequestParam(defaultValue = "20") int limit) {
        return service.list(offset, limit);
    }

    @GetMapping("/{id}")
    public User get(@PathVariable Long id) {
        return service.find(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND));
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public User create(@RequestBody @Valid UserDto dto) {
        return service.create(dto);
    }

    @PutMapping("/{id}")
    public User replace(@PathVariable Long id, @RequestBody @Valid UserDto dto) {
        return service.replace(id, dto);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}

```

Best Practices

- **Use nouns in URLs, verbs in HTTP methods.** Example: `POST /users` to create a user.
- **Pluralize collections.** Example: `/users` for the user collection.
- **Return appropriate status codes.** 201 for creation, 204 for delete, 404 for missing.
- **Include a body on errors.** A JSON error response with `code`, `message`, and `details` beats just a status code.

- **Validate input.** Reject bad requests with 400 before they hit business logic.
 - **Use HTTPS.** Always. Even internal APIs benefit.
 - **Version your API.** You'll need to evolve it without breaking clients.
 - **Document with OpenAPI/Swagger.** Generated docs match the code better than hand-written ones.
 - **Add request IDs.** Every response should carry a unique ID for tracing.
-

Common Gotchas

- Returning 200 OK with an error message in the body. Use proper status codes.
 - Putting actions in URLs (`/users/42/activate`). Better: `POST /users/42/activations` or `PATCH /users/42` with a status field.
 - Mixing pagination styles in the same API. Pick offset or cursor and stick with it.
 - Forgetting CORS. Browsers block cross-origin requests unless your server sets `Access-Control-Allow-Origin` correctly.
 - Coupling your API contract to internal models. Use DTOs to separate the two.
 - Returning 500 for client errors. A malformed request is 400, not 500.
 - Caching POST responses. POST is not cacheable by default. Use GET for safe lookups.
 - Trailing slashes. `/users/` and `/users` can be different routes depending on the framework. Pick one convention and stick with it.
 - Putting sensitive data in query strings. Query strings get logged. Use the body or headers.
-

Quick Reference

Concept	Key fact
Resource	Identified by URL, manipulated by HTTP methods
GET	Safe, idempotent, returns a representation
POST	Not safe, not idempotent, used for creation
PUT	Idempotent, replaces the whole resource
PATCH	Partial update, often idempotent
DELETE	Idempotent, removes the resource
2xx / 4xx / 5xx	success / client error / server error
Stateless	No server-side session between requests
Bearer token	Most common auth (<code>Authorization: Bearer ...</code>)
Idempotency-Key	Header that makes retries safe for POST
Pagination	offset/limit or cursor-based
Versioning	URL (<code>/v1/...</code>) is the easiest to debug